

Stacking Text Classifiers with Collection-Order Label Context for Harmful-Content Detection in Bangla, English and Banglish

Team IUT BakeNekos

System report for the HerWILL X UAP Safe Social Media Datathon 2026 (Kaggle) · 30 September 2026

Abstract

We describe our system for the HerWILL X UAP Safe Social Media Datathon 2026, a shared task that asks for social-media posts in Bangla, English and code-mixed Banglish to be classified as explicitly toxic, subtly toxic or neutral, scored by macro F1. Our system is a two-level stack. At level 1, four text classifiers are trained on one fixed 5-fold split: Gemma 4 31B fine-tuned with QLoRA as a label-token classifier, multilingual-e5-large, Qwen3-32B with a classification head, and a TF-IDF logistic regression, averaged over ten seed runs (45 fine-tuned GPU fold models). At level 2, a gradient-boosting stacker, bagged over five seeds, combines their out-of-fold probabilities with features that summarise the labels of training posts adjacent in the dataset's collection order. We identify this structure in the data: posts adjacent in id order share a label 61.5% of the time against 40.6% by chance. Per-class multipliers then calibrate the output for macro F1. The system reaches 0.7551 out-of-fold macro F1 (0.7547 on held-out folds) and 0.7554 on the private leaderboard, whereas the best text-only ensemble reaches 0.631. We compare 27 level-1 model variants with per-class and per-script scores, report stacker ablations, document a target-leakage failure we detected, and show that the public leaderboard most likely covers a contiguous, harder region of the id range, which explains why every stack scored about 0.015 below its cross-validation there. We close with the implications for deployment: order-based features do not exist in live moderation, and Bangla posts are served measurably worse than English ones.

Keywords: harmful-content detection; implicit toxicity; Bangla; code-mixing; stacked generalisation; QLoRA; data leakage; macro F1.

1 Introduction

Automatic detection of harmful language is a long-standing problem, and its hardest cases are rarely the explicit ones. Offensive and hateful language are easily confused [1], implicit or coded hate often escapes keyword-level models [2], and classifiers can inherit social biases from their training labels [3, 4]. These difficulties grow in low-resource and code-mixed settings. Bangla has comparatively few labelled resources for this task [5], and social-media users freely mix Bangla script, English and romanised Bangla ("Banglish") [6].

The HerWILL X UAP Safe Social Media Datathon 2026 targets exactly this setting. Each of 59,772 posts is labelled as *explicitly toxic or hateful* (class 0), *subtly toxic* (class 1: sarcasm, coded language) or *non-toxic / neutral* (class 2). Submissions are ranked by macro F1 on a hidden 60% of the test set. Only the provided data may be used, and the competition window was about 24 hours.

This report documents our complete solution and the experiments behind it. Our contributions are:

- A reproducible two-level system with a strict validation contract: one pinned 5-fold split and a fixed output format shared by more than fifty training runs on twelve Kaggle accounts and a university GPU cluster (Section 4).
- A comparison of 27 level-1 model variants (two TF-IDF models, ten encoder architectures, fine-tuned and prompted large language models), with per-class and per-script scores (Section 5).
- The identification of a collection-order label signal, leakage-safe features that exploit it, and a case study of a leaking feature that we caught and discarded (Section 6).

- Ablations of the stacker, and an analysis of the gap between the public leaderboard and cross-validation. The analysis led us to keep selecting by cross-validation, which matched the private leaderboard to within 0.0003 (Sections 7 and 8).
- A discussion of deployable accuracy and of the disparity between Bangla and English posts (Section 9).

2 Task and data

2.1 Task definition and metric

Given the text of a post, predict one of three classes. The metric is the unweighted mean of per-class F1, $F1_{\text{macro}} = (1/3) \sum_c 2P_c R_c / (P_c + R_c)$, where P_c and R_c are the precision and recall of class c . Because every class weighs one third, the neutral class, only 12.3% of the training rows, carries as much of the score as the explicit class, which covers half of them. The test set is split into a public part (40%, shown during the competition) and a private part (60%, used for the final ranking). Each team may select up to two submissions for the private ranking.

2.2 Data statistics

Table 1: Dataset statistics, measured on the competition files.

Statistic	Value
Training posts / test posts	47,817 / 11,955
Class shares in train (0 explicit / 1 subtle / 2 neutral)	49.7% / 38.0% / 12.3%
Posts containing Bangla script (train / test)	42.2% / 42.0%
Words per post: median (train / test), mean, 99th percentile (train)	8 / 8, 11.9, 58
Posts containing a URL / an @handle (train)	0.0% / 0.3%
Duplicate text groups in train after cleaning (rows); groups with conflicting labels	468 (1,100); 55
Test posts whose cleaned text occurs in train (with one unambiguous label)	281 (272)
id range (train and test together)	0–59,771, contiguous

Posts are short (a median of eight words) and informal. Emojis, repeated punctuation and capitalisation are common, and they often carry the sarcasm that defines class 1. Some representative, mild training examples: “Where’s Greta!!? We need Greta to stop the snow!! Greta, please help us!!” (subtle); “Next fight: Jake Paul vs the play station boxing game champion” (subtle); “Real talent thakle eto marketing lagto na” (Banglish, *if there were real talent, it wouldn’t need this much marketing*; labelled neutral); “Kinda sad to see what a joke burr has turned into.” (neutral). The last two show how thin the boundary between the subtle and neutral classes can be.

2.3 Script and class distribution

Table 2: Class distribution by script. The two groups have quite different label mixes, so script is informative in itself.

Posts	Share of train	Explicit	Subtle	Neutral
Containing Bangla script	42.2%	59.2%	33.7%	7.2%
English or Banglish (no Bangla script)	57.8%	42.8%	41.1%	16.1%
All	100%	49.7%	38.0%	12.3%

2.4 Label consistency

Labels are noisy, as one would expect for subtle toxicity. Among training posts with a near-identical twin (cosine similarity ≥ 0.99), the two share a label only 84% of the time, and about 3% of test posts have such a twin in the training set. When near-identical texts disagree one time in six, macro F1 far above 0.8 is implausible, so we set a working target of 0.75–0.80. This ceiling later served as a sanity check (Section 6.3).

2.5 Collection order

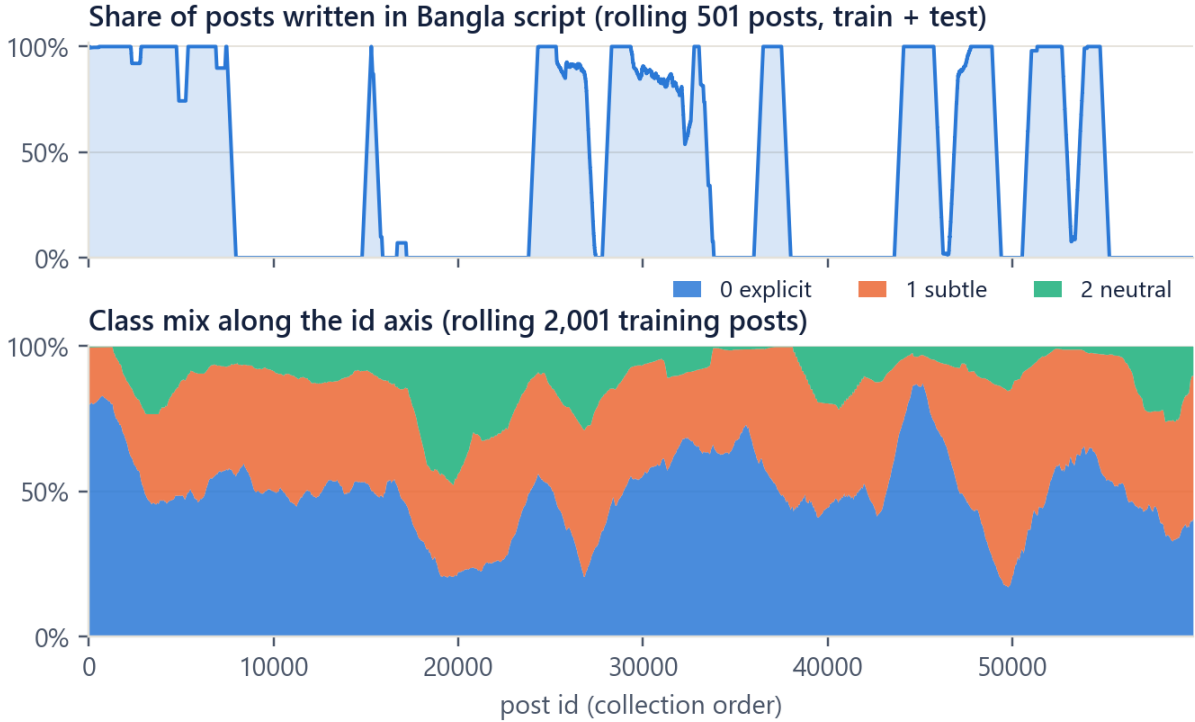


Figure 1: Top: share of posts in Bangla script along the id axis (rolling window of 501 posts, train and test together). Bottom: class mix of training posts along the id axis (rolling window of 2,001 posts). Both vary in long, smooth stretches.

The `id` column is contiguous from 0 to 59,771 across the training and test files together, and the test posts occupy what look like random positions within it. Figure 1 shows that script and class mix both change in long stretches along this axis. Ids 0–6,000, for example, are 97% Bangla, while ids 18,000–24,000 are almost entirely English. We read the `id` as the collection order of the posts, and Section 6 shows that it carries a strong label signal.

3 Related work

Harmful and implicit toxicity. Early work separated hate speech from merely offensive language [1]. Later benchmarks target implicit and coded hate [2] and show that classifiers can encode annotator and dialect bias [3, 4]. Our class 1 (subtle toxicity) is closest to implicit hate.

Bangla and code-mixed NLP. Monolingual and regional encoders such as BanglaBERT [7], MuRIL [8] and IndicBERTv2 [9] were built for Bangla and other Indic languages. Bangla hate-speech detection has been studied with such encoders [5], and code-switched text poses its own difficulties [6].

Multilingual encoders and LLM fine-tuning. Our encoder baselines are XLM-R [10], its Twitter-adapted variant XLM-T [11], mDeBERTa-v3 [12] and multilingual E5 [13]. For decoder LLMs we use low-rank adaptation [14] on 4-bit quantised weights [15], applied to Qwen3 [16] and Gemma 4 [17], and run prompted baselines with vLLM [18]. All models are trained with Hugging Face Transformers [19].

Ensembles and semi-supervision. Stacked generalisation [20] trains a second-level model on out-of-fold predictions of first-level models. We use gradient-boosted trees [21] as the stacker and compare implementations [22, 23, 24, 25]. Pseudo-labelling [26] adds confidently predicted unlabelled examples to

training.

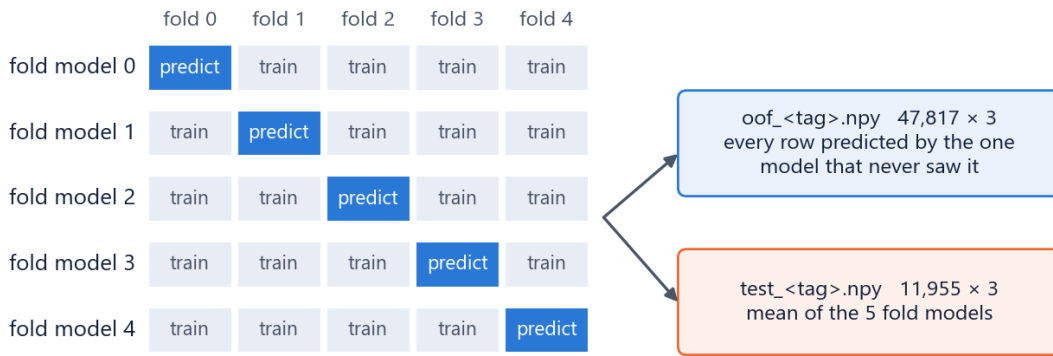
Leakage and metric calibration. Target leakage, where a feature encodes the label through the way it is computed, is a well-known failure of predictive modelling [27]. Our Section 6.3 describes a textbook instance. For F1-type metrics, the decision rule should be tuned rather than taken as the argmax of calibrated probabilities [28]. We tune per-class multipliers accordingly.

4 Experimental protocol

4.1 Pinned cross-validation and the output contract

All experiments share one split. Rows are sorted by id, and folds come from `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` [25], giving fold sizes of 9,564 / 9,564 / 9,563 / 9,563 / 9,563 (asserted in every script). Every training run, whatever the model or machine, must write three artifacts: `oof_<tag>.numpy` ($47,817 \times 3$ probabilities, each row predicted by the fold model that did not train on it), `test_<tag>.numpy` ($11,955 \times 3$, the mean of the five fold models) and `log_<tag>.json` (configuration, per-fold scores, training history).

47,817 training posts, sorted by id, split once: `StratifiedKFold(5, shuffle, seed 42)`



Every model (and every seed) follows the same contract, so any set of models can be stacked without leakage.

Figure 2: The output contract. Each fold model predicts only its held-out fold, so any combination of runs can be stacked without leaking labels.

The contract (Figure 2) is what makes stacking safe: a second-level model learns how far to trust each first-level model from out-of-fold predictions only. It also allowed runs from many machines and seeds to be merged, averaged and compared mechanically.

4.2 Reported scores and model selection

We report four quantities. *OOF* is the macro F1 of the argmax of the 47,817 out-of-fold predictions. *Tuned* is the OOF macro F1 after three per-class probability multipliers have been chosen to maximise it (Section 6.5). *Held-out* fits those multipliers on folds 0–2 only and scores folds 3–4, an honest estimate of the tuned score. *Public* and *private* are the leaderboard scores. All model selection was done on OOF and held-out scores. The public leaderboard covers only about 4,800 posts, and Section 8.1 shows that it is not a random sample.

4.3 Preprocessing

Preprocessing is deliberately minimal and identical for every model:

```
URL, USER, WS = re.compile(r"https?://\S+|www\.\S+"), re.compile(r"@w+"), re.compile(r"\s+")
clean = lambda s: WS.sub(" ", USER.sub("<user> ", URL.sub("<url> ",
    unicodedata.normalize("NFKC", str(s))))).strip()
```

NFKC normalisation unifies alternative Unicode encodings of the same Bangla glyphs. URLs and user handles are replaced by placeholder tokens, and whitespace is collapsed. Emojis, punctuation and casing are kept, because they carry much of the sarcasm signal and modern subword tokenizers represent them. BanglaBERT models additionally receive the csebuetnlp normaliser they were pre-trained with [7].

4.4 Compute

Table 3: Compute used. Kaggle runs were pushed and tracked with small scripts (push.py, pull.py, a run registry); cluster runs used a Slurm work queue with one fold per GPU and scripts that verify shapes, folds and scores before hand-off.

Resource	Hardware	Used for
Kaggle notebooks, 12 team accounts	2 × NVIDIA T4 (16 GB) per session, at most 2 sessions per account	TF-IDF, base and large encoders, the Qwen3-14B QLoRA pilot, submission notebooks
University cluster (Slurm)	NVIDIA H200, A100 80 GB, RTX 6000 Ada, RTX A6000, RTX PRO 4000 Blackwell	Gemma 4 31B, Qwen3-32B/14B, e5-large, MuRIL-large, BanglaBERT-large, XLM-R-large seeds, vLLM judges
Laptop CPU	—	stacking, blending, calibration, analysis (about 36 s per 5-fold stacker run)

The software stack on the cluster was torch 2.12, transformers 5.6, peft 0.19 and bitsandbytes 0.50, and stacking used scikit-learn 1.9. The whole project, from reading the brief to the final submission, took about 23.5 hours. Appendix C gives the timeline.

5 Level-1 text classifiers

5.1 TF-IDF baseline

Word 1–2-grams and character 2–5-grams (within word boundaries) feed a class-balanced logistic regression with $C = 4$. It reaches 0.5219 OOF and was our first submission (public 0.54091).

5.2 Transformer encoders

We fine-tune XLM-R base and large [10], Twitter-XLM-R [11], mDeBERTa-v3 [12], MuRIL base and large [8], BanglaBERT base and large [7], IndicBERTv2 [9] and multilingual-e5-large [13], each with a linear classification head. The shared recipe is AdamW with weight decay 0.01, a cosine schedule with 6% warm-up, 3 epochs, a maximum length of 128 tokens and mixed precision. The loss is class-weighted cross-entropy with weights $w_c = N / (3 n_c)$. The learning rate is $2 \cdot 10^{-5}$ for base models and 10^{-5} for large ones. Appendix A lists every run.

5.3 An effective-step bug on two-GPU machines

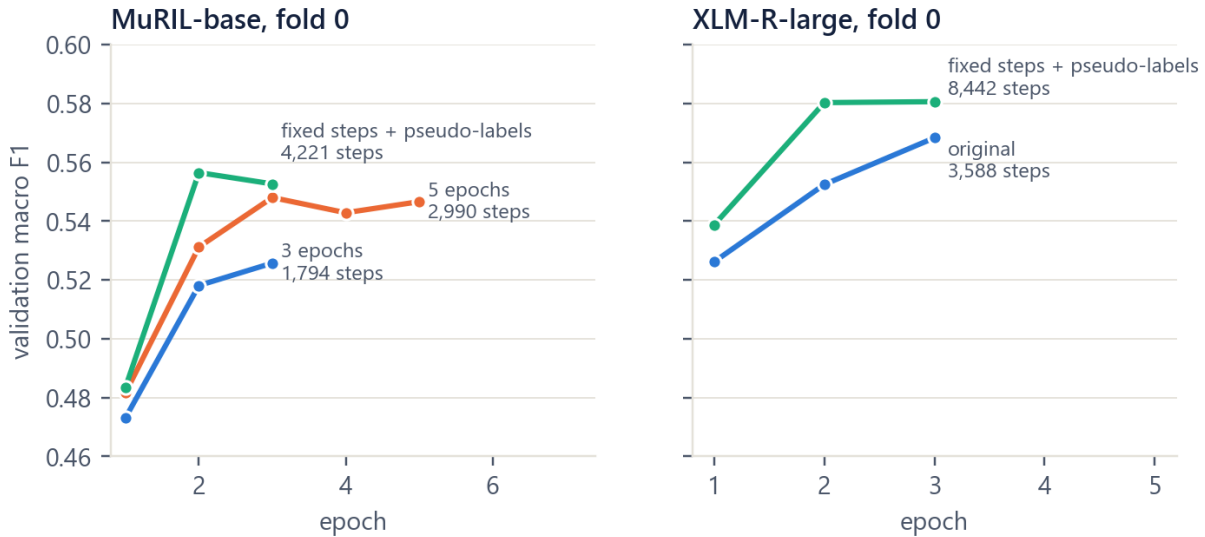


Figure 3: Validation macro F1 by epoch on fold 0. Left: MuRIL-base with 3 epochs, 5 epochs, and the restored step count plus pseudo-labels. Right: XLM-R-large before and after restoring the step count.

Kaggle's two-GPU machines make the Hugging Face Trainer wrap the model in DataParallel, which doubles the effective batch and halves the number of optimizer steps. Our first XLM-R-large runs took about 3,500 steps instead of the planned 7,200, and their validation F1 was still rising at the last epoch (Figure 3). A MuRIL run with 5 epochs improved OOF by +0.014 (0.5345 $\rightarrow$ 0.5488) from the extra steps alone. From then on, per-GPU batch sizes were halved to restore the intended step count. The cluster runs used one GPU per process and were not affected.

5.4 Decoder LLMs with QLoRA

Qwen3 with a classification head. Qwen3-14B and Qwen3-32B [16] are loaded in 4-bit NF4 [15], and LoRA adapters [14] ($r = 16$, $\alpha = 32$, dropout 0.05) are attached to all attention and MLP projections (q, k, v, o, gate, up, down), together with a sequence-classification head. Training runs for one epoch with an effective batch of 32 and learning rates of 10^{-4} (14B) and $5 \cdot 10^{-5}$ (32B), using class-weighted cross-entropy on fp32 logits, length-bucketed batches and gradient checkpointing. The Qwen tokenizer splits Bangla into about 2.7 times more tokens than XLM-R (a mean of 54 against 20), so the maximum length is 192. A two-fold pilot of Qwen3-14B on Kaggle T4s (5.2 h per fold, fp16 compute) had to clear a pre-registered gate before the remaining folds were launched: beat XLM-R-large on the same rows. It scored 0.5919 against 0.5650.

Gemma 4 as a label-token classifier. transformers 5.6 has no sequence-classification class for Gemma 4 [17]. Instead of adding a new head, we cast classification as next-token prediction:

```
Classify this social-media post (Bangla, English or Banglish). 0 = explicitly toxic or hateful,
1 = subtly toxic (sarcasm, coded language), 2 = non-toxic or neutral.
Post: <cleaned post, truncated so that the suffix always survives>
Label:
```

The logits of the tokens “0”, “1” and “2” at the final position are the class scores, and training minimises class-weighted cross-entropy over those three logits. Left padding keeps the answer slot at the last position of every row. The model reuses its pre-trained output layer, so no parameters start from scratch. LoRA settings match the Qwen runs, with a learning rate of $5 \cdot 10^{-5}$, one epoch and a maximum length of 128 (the Gemma tokenizer's 99th percentile is 77 tokens). We trained four seeds (42, 2, 3, 4) and an instruction-tuned variant.

5.5 Prompted LLM judges

As a zero-training reference, Gemma 4 31B-it (FP8 weights) and Qwen3-32B-AWQ were prompted with nine labelled examples through vLLM [18]. Class probabilities came from the log-probabilities of the three label tokens, and the few-shot examples were excluded from scoring.

5.6 Neighbour-text context

Because neighbouring posts are related (Section 6), we also trained text models that read them. The context-aware e5-large receives the text pair (post, “previous: <previous post> | next: <next post>”) with a maximum length of 256, and a TF-IDF variant appends the neighbouring posts’ text. These models use neighbours’ *text* only, never their labels.

5.7 Fold-blind pseudo-labelling

Pseudo-labelling [26] adds confidently predicted test posts to the training data. To keep the OOF estimate honest, our pseudo-labels are *fold-blind*: the pseudo-labels used by fold f ’s model come from a stacker that was trained without fold f . Otherwise the pseudo-labels would carry information about fold f ’s labels into its own model. Version 1 kept 55–57% of test posts per fold, where the stacker’s OOF accuracy was at least 85%. Version 2, built from a stronger stacker, kept 61–64%.

Table 4: Pseudo-labelling on XLM-R-large (the same rows). The gain mixes two changes. The MuRIL runs separate them: +0.014 from restoring the steps and roughly +0.01 from the pseudo-labels.

XLM-R-large, macro F1 on fold	1	2	3	4
seed 42 (original step count)	0.5617	0.5636	0.5094	0.5714
seed 2 (original step count)	0.5602	0.5629	0.5601	0.5573
+ pseudo-labels v1, restored step count	0.5794	0.5768	0.5877	0.5804

5.8 Results

Table 5: All level-1 models (5-fold OOF macro F1; seed runs are averaged before scoring). “Tuned” applies per-class multipliers fitted on OOF. The last two columns split OOF macro F1 by whether the post contains Bangla script (Other = English and Banglish). Bold rows are the members of the final stack. PL = pseudo-labels.

Model	Seeds	OOF	Tuned	F1 expl.	F1 subtle	F1 neutral	Bangla	Other
<i>Classical</i>								
TF-IDF + logistic regression	1	0.522	0.525	0.647	0.498	0.420	0.443	0.542
TF-IDF + neighbour text	1	0.566	0.567	0.685	0.543	0.470	0.493	0.582
<i>Encoders, Kaggle T4 ×2</i>								
mDeBERTa-v3-base	1	0.489	0.529	0.664	0.382	0.420	0.402	0.510
BanglaBERT	1	0.528	0.553	0.656	0.473	0.455	0.477	0.536
MuRIL-base (3 ep)	1	0.535	0.562	0.672	0.456	0.476	0.478	0.546
MuRIL-base (5 ep)	1	0.549	0.563	0.661	0.504	0.482	0.486	0.565
MuRIL-base + PL	1	0.558	0.571	0.675	0.505	0.496	0.496	0.574
XLM-R-base	1	0.537	0.564	0.668	0.473	0.471	0.457	0.563
Twitter-XLM-R-base	1	0.557	0.578	0.667	0.507	0.497	0.469	0.593
Twitter-XLM-R-base + PL	1	0.576	0.587	0.683	0.528	0.517	0.483	0.610
IndicBERTv2	1	0.553	0.574	0.670	0.505	0.484	0.485	0.574
IndicBERTv2 + PL	1	0.561	0.579	0.681	0.509	0.493	0.490	0.582
XLM-R-large (seeds 42, 1337, 7)	3	0.579	0.590	0.689	0.521	0.527	0.487	0.611
XLM-R-large (seed 2)	1	0.562	0.583	0.681	0.501	0.502	0.468	0.595
XLM-R-large + PL	1	0.581	0.591	0.693	0.524	0.526	0.487	0.613
<i>Encoders, cluster</i>								
BanglaBERT-large	2	0.561	0.570	0.661	0.531	0.490	0.500	0.577
MuRIL-large	3	0.585	0.594	0.688	0.545	0.523	0.505	0.613
multilingual-e5-large	3	0.600	0.604	0.700	0.551	0.548	0.492	0.640
multilingual-e5-large + PL v2	2	0.602	0.610	0.706	0.547	0.552	0.492	0.642
multilingual-e5-large + neighbour text	3	0.626	0.631	0.714	0.572	0.592	0.524	0.663
<i>LLMs, fine-tuned</i>								
Qwen3-14B QLoRA (Kaggle T4)	1	0.592	0.610	0.694	0.533	0.549	0.493	0.625
Qwen3-14B QLoRA (cluster)	2	0.596	0.613	0.701	0.533	0.556	0.493	0.630
Qwen3-32B QLoRA	2	0.596	0.611	0.698	0.538	0.554	0.493	0.630
Gemma 4 31B-it, label tokens	1	0.610	0.620	0.707	0.549	0.574	0.498	0.647
Gemma 4 31B QLoRA, label tokens	4	0.625	0.636	0.717	0.563	0.594	0.516	0.662
Gemma 4 31B + PL v2	2	0.624	0.635	0.724	0.554	0.593	0.510	0.661
<i>LLMs, prompted</i>								
Gemma 4 31B-it, 9-shot judge	1	0.553	0.559	0.618	0.533	0.508	0.476	0.572

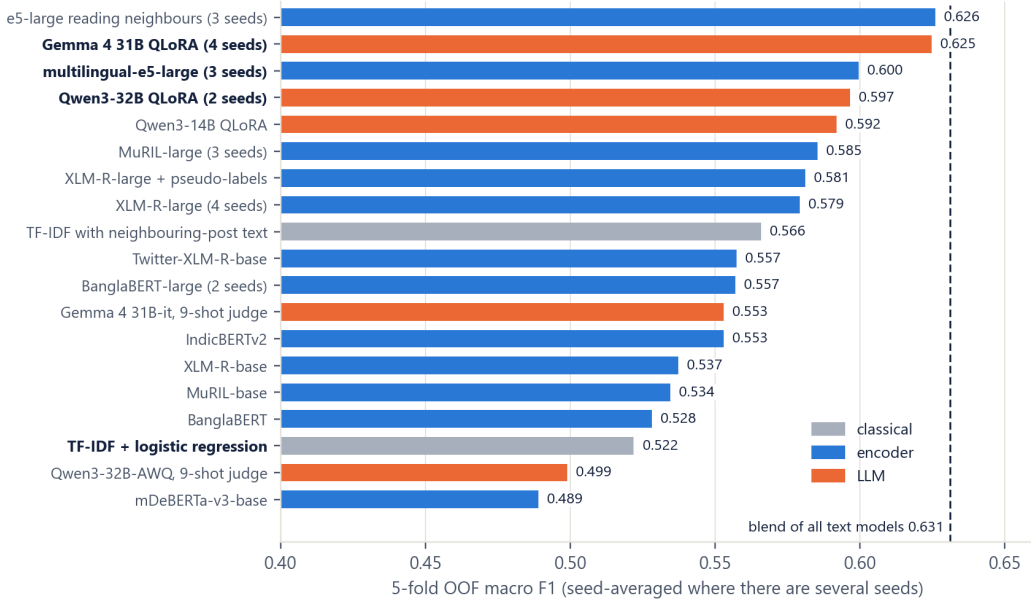


Figure 4: Level-1 models ranked by OOF macro F1. The dashed line marks the text-only blend of all models (0.631).

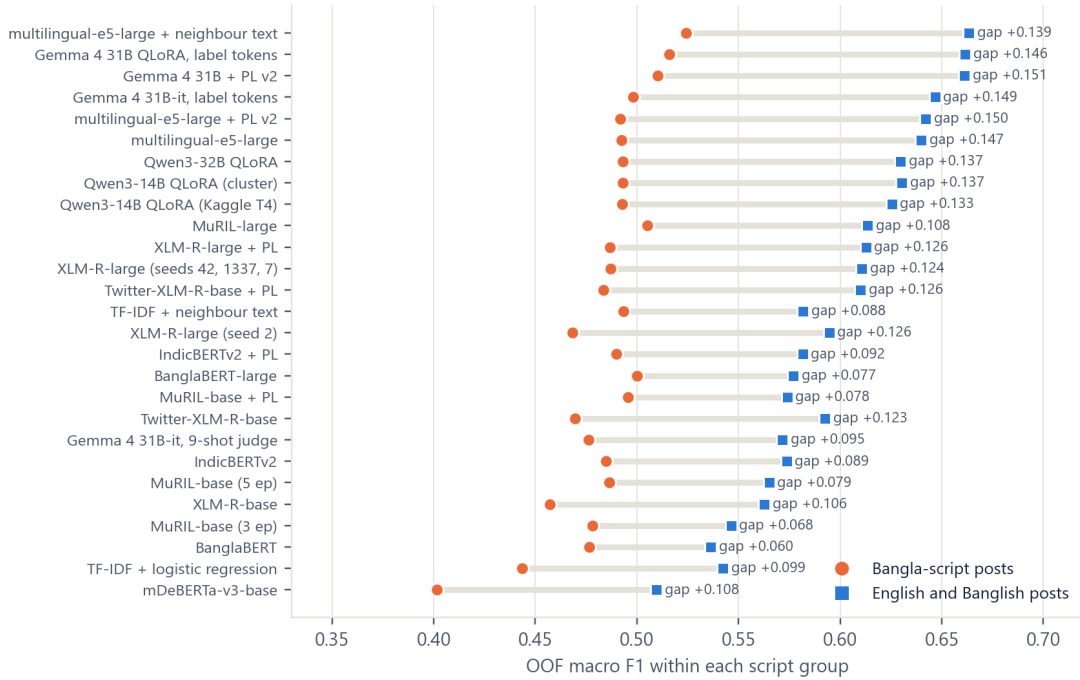


Figure 5: OOF macro F1 of each level-1 model on posts with Bangla script (circles) and on English and Banglish posts (squares). Every model is weaker on Bangla.

Table 5 and Figures 4–5 summarise the level-1 models. Five observations shaped the final system:

- **Model family mattered more than size.** Qwen3-32B (0.597) is no better than Qwen3-14B (0.596), while Gemma 4 31B reaches 0.625, about 0.03 higher.
- **Fine-tuning beats prompting by a wide margin.** The same Gemma checkpoint scores 0.553 as a 9-shot judge and 0.625 fine-tuned, and the Qwen3-32B judge reaches only 0.499.
- **A strong encoder is cost-effective.** multilingual-e5-large reaches 0.600 in about 12 minutes per fold, against roughly 3 hours per fold for Gemma.
- **Every model is weaker on Bangla** (Figure 5), by 0.06–0.15 macro F1. The Bangla-specific encoders have the smallest gaps but are weaker overall.
- **Reading neighbouring text helps text models:** +0.026 for e5-large (0.600 → 0.626) and +0.044 for TF-IDF (0.522 → 0.566).

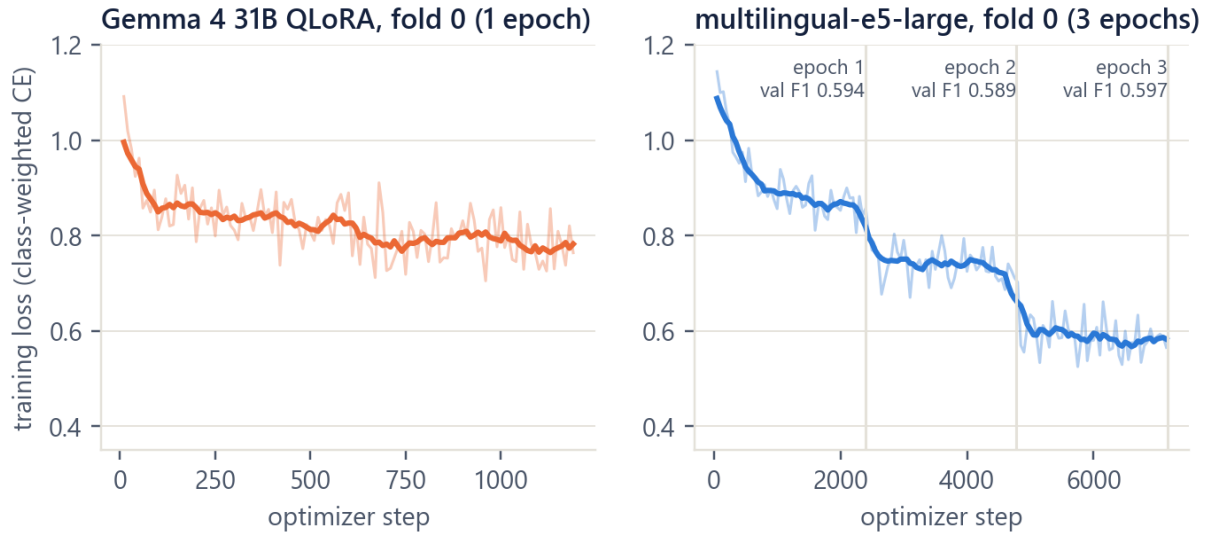


Figure 6: Training loss on fold 0 (thin: logged values; thick: moving average). Left: Gemma 4 31B, one epoch. Right: multilingual-e5-large, three epochs, with validation macro F1 at each epoch boundary.

Figure 6 shows typical training curves. Gemma’s loss falls quickly and then flattens within one epoch. The e5-large loss drops at every epoch boundary while its validation F1 stays flat (0.594, 0.589, 0.597), so longer training would mostly memorise the training posts.

5.9 Text-only blending

Table 6: Weighted blends of text models. Blending adds about +0.006 over the best single model; text alone plateaus near 0.63.

Blend (weights fitted on OOF, then tuned)	OOF tuned	Held-out
v1: TF-IDF + BanglaBERT	0.5632	0.5738
v2: + MuRIL	0.5766	0.5788
v3: + XLM-R-base, Twitter-XLM-R	0.5934	0.5985
v4: + XLM-R-large (6 models)	0.5984	0.5913
v5: 9 models including XLM-R-large + PL	0.5998	0.6022
All text models, including the cluster LLMs	0.6307	0.6373

6 Level-2 stacking with collection-order label context

6.1 The signal

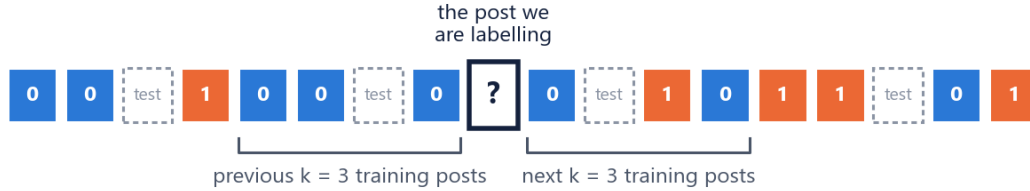


Figure 7: Probability that two training posts share a label, as a function of their distance in id order. The dashed line is the rate under independence, $\sum_c p_c^2 = 0.406$.

Training posts that are adjacent in id order share a label 61.5% of the time, against 40.6% if labels were independent (Figure 7). The excess decays smoothly with distance (0.555 at 10 rows, 0.505 at 100), which points to a slowly drifting mix of sources or threads rather than label-sorted blocks. A 5-nearest-neighbour vote over id alone, which reads no text at all, scores 0.600 OOF, as much as the best single text model available at the time. Test posts sit at random positions among the training posts, so a test post has labelled neighbours on both sides, just as a training post does.

6.2 Features

Posts in id order. Coloured = training post (its label shown); dashed = test post (no label).



Features for '?': the class mix of the previous k, next k and surrounding k training posts, for k = 1, 2, 5, 10, 25, 100, plus surrounding windows of 250, 500, 1,000 and 2,500 posts, and the id gaps to the nearest training posts. Windows stop strictly before and start strictly after the post, so a training post never sees its own label.

Figure 8: Construction of the neighbour-label features for one post. Only training posts carry labels; the windows skip test posts and never include the post itself.

Let $t_1 < \dots < t_n$ be the training ids and $\mathbf{y}_j$ the one-hot label of training post j . For a post with id i , let a be the number of training ids strictly below i and b the index of the first training id strictly above it. The *previous-k* feature is the mean of $\mathbf{y}_{a-k+1}, \dots, \mathbf{y}_a$, the *next-k* feature is the mean of $\mathbf{y}_b, \dots, \mathbf{y}_{b+k-1}$, and *both-k* pools the two windows. All are computed with prefix sums in $O(1)$ per post. Table 7 lists the full feature set; Figure 8 shows the construction.

Table 7: Features of the final stacker.

Group	Columns	Definition
Text	12	OOF probabilities of the 4 members (3 classes each); 5-fold mean for test posts
Neighbour labels	54	previous-k, next-k and both-k class mixes for k = 1, 2, 5, 10, 25, 100
Wide windows	12	both-k class mixes for k = 250, 500, 1,000, 2,500
id geometry	3	id gap to the previous and to the next training post; the raw id
Context (label-free)	5	Bangla-script flag; log character length; share of Bangla-script posts among the 5 / 21 / 101 posts centred on the post (train and test together)
Total	86	

6.3 Leakage safeguards and a detected leak

Label-derived features are dangerous [27], so every id feature obeys three rules. (i) Windows end strictly before and start strictly after the post, over training posts only, so a post's own label never enters its own features. (ii) A test post gets exactly the view a training post gets: all training posts around it. (iii) Neighbours' *text-model predictions* are not used. A training neighbour's OOF prediction comes from a model whose training folds contained the current post, so it can carry that post's label. This never happens for a test post, so the feature would behave differently at test time. It also added nothing (+0.0000).

A case study. One candidate feature, the label mix of the post's entire *language run* (a maximal contiguous block of same-script posts) minus the post's own label, raised the stacker to 0.97 OOF. All posts in a run share the same run total, so “total minus own label” differs between them only by their own label: the feature is a leave-one-out target encoding and reveals the answer. An honest leave-one-out majority vote inside a run is only 54–70% accurate. The label-noise ceiling of Section 2.4 made the leak obvious, since 0.97 is far above what 84% twin agreement allows. The feature was discarded before any submission.

6.4 Stacker

The stacker is scikit-learn's HistGradientBoostingClassifier [21, 25] with 600 iterations, learning rate 0.04, 31 leaves, at least 40 samples per leaf, L2 regularisation 1.0, balanced class weights and early stopping on an internal validation split. It is trained on the same five folds as level 1, so its OOF predictions stay honest. Five stackers with different seeds (random_state = fold + 100 · s, s = 0...4) are trained per fold and averaged. The test prediction is the mean of all 25.

6.5 Calibration for macro F1

Balanced class weights make the stacker over-predict the rare neutral class. Following the observation that F1-optimal decisions differ from the argmax [28], we multiply the class probabilities by a vector $\mathbf{a}$ and predict $\text{argmax}_c \mathbf{a}_c p_c$. The vector is found by coordinate ascent over a log-spaced grid of 65 values between $e^{-1.6}$ and $e^{1.6}$, for up to four rounds. The result, normalised, is $\mathbf{a} = (1, 0.905, 0.549)$: neutral is roughly halved. Calibration lifts OOF from 0.7456 to 0.7551. Fitted on folds 0–2 only, it scores 0.7547 on folds 3–4, so it does not overfit.

6.6 Verbatim override

The 272 test posts whose cleaned text occurs in the training data with a single label take that label. This changes 32 predictions and adds +0.0005 in a leave-one-out simulation on the training set.

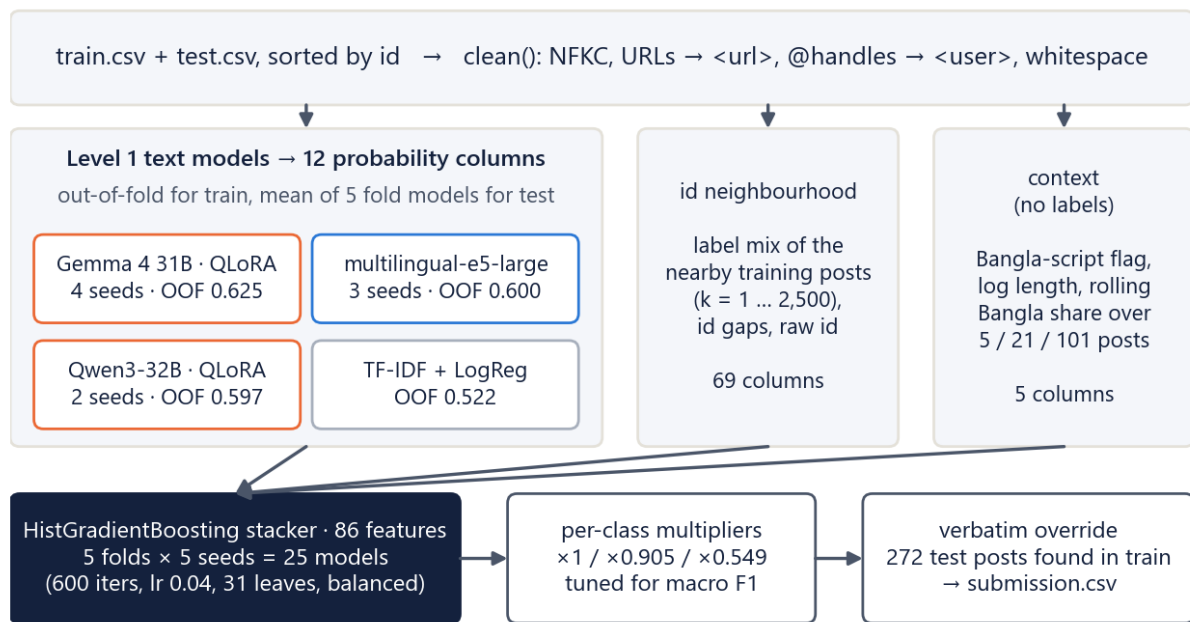


Figure 9: The complete system (stack v9 with a 5-seed bag).

7 Results

7.1 Main results

Table 8: Main results (macro F1). Private-leaderboard scores are from the competition's submission page; n/r = not recorded.

System	OOF raw	OOF tuned	Held-out	Public	Private
TF-IDF + logistic regression	0.5219	0.5249	—	0.54091	n/r
Best single text model (Gemma 4 31B, 4 seeds)	0.625	0.636	—	—	—
Text-only blend of all level-1 models	—	0.6307	0.6373	—	—
Stack v7 (5 encoder members, no LLMs)	0.7114	0.7253	0.7289	—	—
Stack v9 (one stacker seed, Kaggle run)	—	0.7529	0.7542	0.73848	0.75634
Stack v9, 5-seed bag (final submission)	0.7456	0.7551	0.7547	0.74088	0.75543
Stack v10 (+ id-distance kernels; Section 8.4)	—	0.7620	0.7625	0.73957	0.76000

The final system scores 0.7551 OOF, 0.7547 held-out and 0.7554 on the private leaderboard, so cross-validation predicted the private score to within 0.0003. It improves on the best single text model by 0.13 and on the best text-only blend by 0.12, almost all of it from the collection-order features. Per class, it reaches F1 = 0.814 (explicit), 0.710 (subtle) and 0.742 (neutral).

7.2 Development of the stack

Table 9: Evolution of the stack. The v9 range covers different stacker seeds and scikit-learn builds.

Stack	Members and features	OOF tuned	Held-out
v1	wave-1 encoders + TF-IDF; neighbour windows $k \leq 100$; context	0.7000	0.7054
v2	5 text models	0.7071	0.7136
v3	6 text models	0.7110	0.7138
v4	+ mDeBERTa; wide windows 250–2,500	0.7198	0.7214
v5	+ XLM-R-large seed 2	0.7209	0.7233
v6	+ XLM-R-large with pseudo-labels (9 members)	0.7231	0.7266
—	all 12 text models	0.7199	0.7216
v7	best 5: TF-IDF, Twitter-XLM-R, XLM-R-large-PL, MuRIL-PL, IndicBERTv2	0.7253	0.7289
v8	Gemma 4 31B + e5-large + Qwen3-32B + TF-IDF	0.7537	0.7546
v9	v8 members without pseudo-labels, seed-averaged (Gemma $\times 4$, e5 $\times 3$, Qwen $\times 2$)	0.7529–0.7555	0.7542–0.7580
v9 bag	v9 with 5 stacker seeds per fold (final)	0.7551	0.7547
v10	v9 + id-distance kernel label features	0.7620	0.7625

7.3 Ablations

Table 10: Stacker ablations with the final member set. Each row is one 5-fold run with a single GBDT seed per fold; Δ is relative to the first row. Across three stacker seeds the full model scores 0.7537 / 0.7536 / 0.7535 OOF tuned and 0.7549 / 0.7533 / 0.7541 held-out, so differences below about ± 0.002 are noise.

Stacker input	Cols	OOF tuned	Δ	Held-out	Δ	Bangla	Eng./Bangl.
All 86 features (one GBDT seed)	86	0.7537	+0.0000	0.7549	+0.0000	0.719	0.761
Text probabilities only	12	0.6370	-0.1167	0.6354	-0.1195	0.522	0.675
id features only (labels + geometry)	69	0.6650	-0.0886	0.6656	-0.0892	0.669	0.650
– text probabilities	74	0.6635	-0.0902	0.6672	-0.0877	0.665	0.650
– neighbour-label windows	20	0.6937	-0.0600	0.6903	-0.0646	0.655	0.700
– neighbour labels and id geometry	17	0.6435	-0.1102	0.6419	-0.1130	0.544	0.676
– short windows ($k \leq 100$)	32	0.7397	-0.0140	0.7410	-0.0139	0.711	0.743
– wide windows ($k \geq 250$)	74	0.7439	-0.0098	0.7463	-0.0085	0.692	0.756
– id gaps and raw id	83	0.7537	+0.0001	0.7552	+0.0004	0.719	0.760
– context features	81	0.7509	-0.0027	0.7502	-0.0046	0.713	0.758
– Gemma 4 31B	83	0.7433	-0.0104	0.7445	-0.0104	0.703	0.752
– multilingual-e5-large	83	0.7544	+0.0007	0.7578	+0.0029	0.718	0.762
– Qwen3-32B	83	0.7541	+0.0004	0.7554	+0.0006	0.716	0.762
– TF-IDF	83	0.7552	+0.0016	0.7571	+0.0023	0.718	0.763

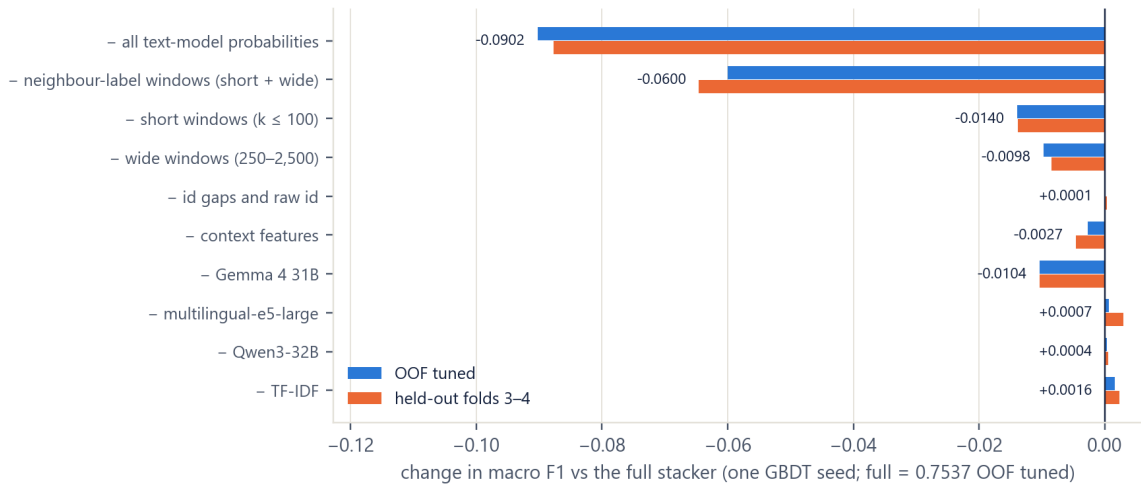


Figure 10: Change in macro F1 when one feature group or one member is removed from the stacker.

Table 10 and Figure 10 decompose the stack. Text probabilities (0.637 alone) and id features (0.665 alone) are complementary: together they reach 0.754. Removing the neighbour-label windows costs 0.060, but the raw id and id gaps still recover much of the signal, because the id itself locates a post in the drifting class mix; removing both costs 0.110. With the windows present, the raw id and gaps are redundant ($\Delta = +0.0001$). Short windows matter more than wide ones (-0.014 against -0.010), and the context features add 0.003. Among the members only Gemma is individually important (-0.010). Removing e5-large, Qwen3-32B or TF-IDF alone changes the score by at most $+0.0015$, within noise: given Gemma, they are largely redundant. The member set was fixed earlier on evidence of the same size (Table 11), so this is a small, honest inefficiency rather than a loss.

7.4 Member selection

Table 11: More members are not better. Gradient boosting over many correlated probability columns spends splits on small, noisy differences. v7 was chosen among five candidate sets, so about 0.002 of its score may be selection luck.

Member set (+ id features)	OOF raw	OOF tuned	Held-out
v6: 9 members	0.7085	0.7231	0.7266
all 12 text models	—	0.7199	0.7216
v7: best 5 members (no LLMs)	0.7114	0.7253	0.7289
Gemma 4 31B + TF-IDF	0.7412	0.7533	0.7546
v8: Gemma + e5-large + Qwen3-32B + TF-IDF	0.7424	0.7537	0.7546
11 members including Gemma	0.7358	0.7501	0.7507

7.5 Error analysis

Table 12: Per-class OOF results of the final system (after calibration). Accuracy is 0.765.

Class	Precision	Recall	F1	Support
0 explicit	0.820	0.807	0.814	23,756
1 subtle	0.705	0.715	0.710	18,163
2 neutral	0.735	0.748	0.742	5,898
Macro average	0.753	0.757	0.755	47,817

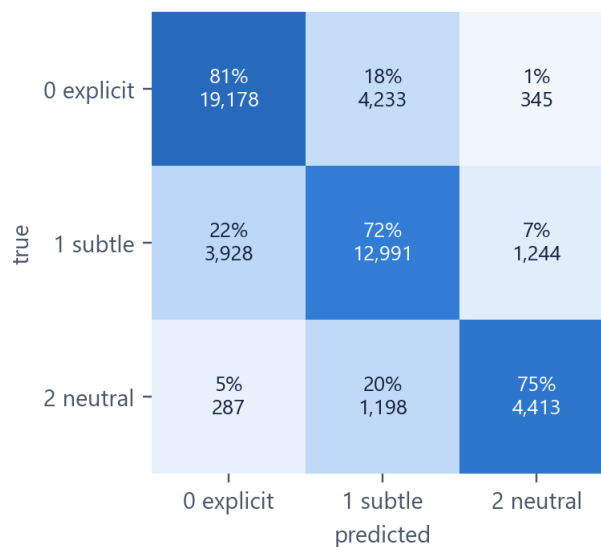


Figure 11: Confusion matrix of the final system on OOF predictions (row-normalised; counts below).

Most errors lie on the explicit-subtle border (4,233 explicit posts predicted subtle and 3,928 subtle posts predicted explicit; Figure 11). Neutral posts are confused mainly with subtle ones (1,198), which matches the ambiguity visible in the examples of Section 2.2. Explicit and neutral are rarely mixed (345 and 287). By script, the system scores 0.7245 on Bangla-script posts and 0.7610 on English and Banglish posts.

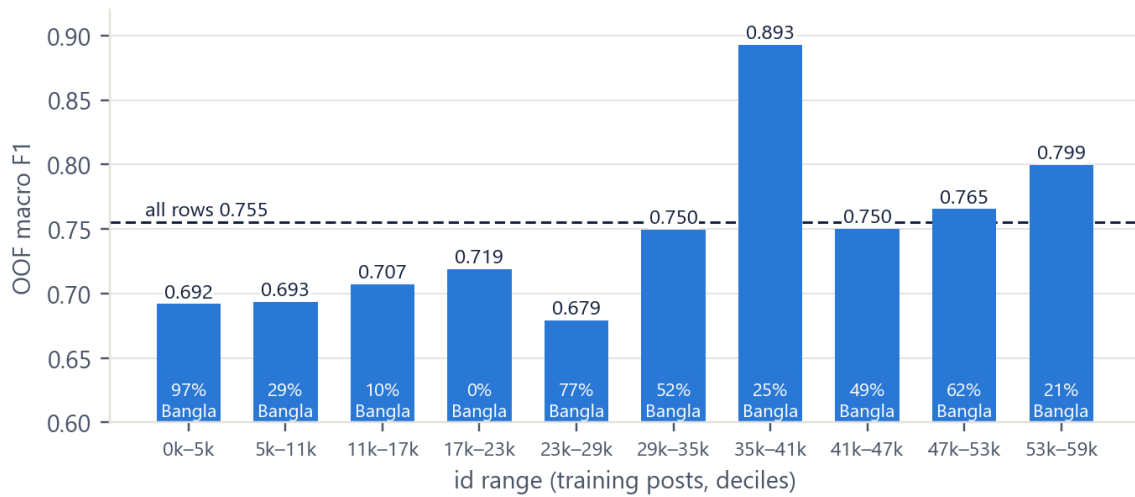


Figure 12: OOF macro F1 of the final system in each tenth of the id range; the share of Bangla-script posts is printed inside each bar.

Figure 12 breaks the score down by id region. Bangla-heavy regions are the hardest (0.68–0.69), while the region 35k–41k reaches 0.89, where neighbouring labels are unusually consistent.

7.6 Leaderboard submissions

Table 13: All Kaggle submissions. Our best public score (0.74302) ranked third on the public leaderboard at the deadline.

Time (UTC)	Submission	OOF	Public	Private
27 Sep 12:49	TF-IDF + logistic regression	0.5219	0.54091	n/r
27 Sep 12:50	TF-IDF, per-class calibrated	0.5249	0.54629	n/r
28 Sep 07:56	Stack v8 (pseudo-labelled members; Gemma re-run on Kaggle)	0.7524	0.73843	n/r
28 Sep 09:31	Stack v9 (one stacker seed)	0.7529	0.73848	0.75634
28 Sep 10:52	Stack v9 + TF-IDF-context member, 5-seed bag (selected)	0.7546	0.74302	0.75189
28 Sep 10:52	Stack v9, 5-seed bag (selected; final)	0.7551	0.74088	0.75543
28 Sep 10:57	Stack v10, id-distance kernels	0.7620	0.73957	0.76000
after deadline	grand6: mean of six stackers (late submission)	0.7576	0.74324	0.75583

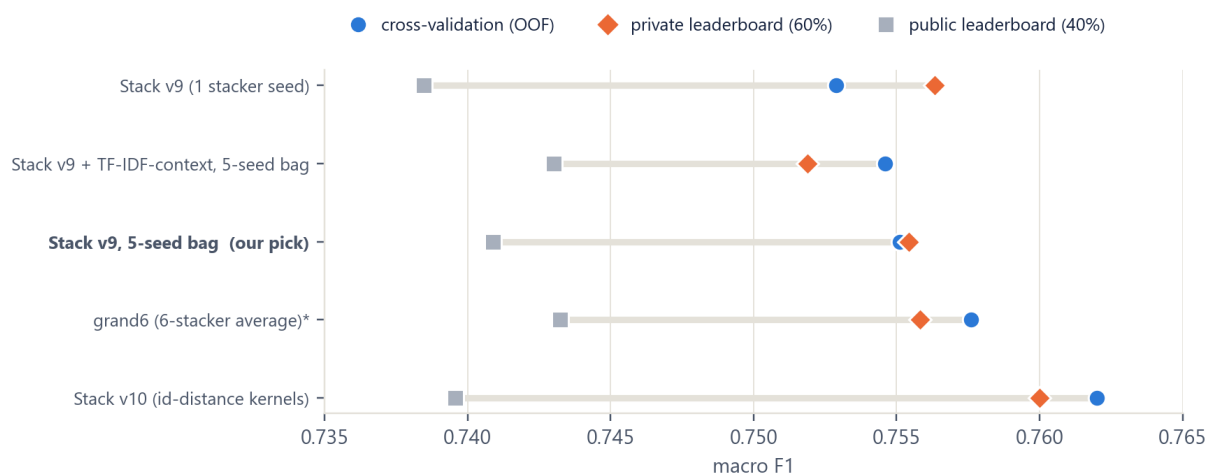


Figure 13: Cross-validation, private and public scores of the stack submissions. *Late submission.

Figure 13 shows the pattern across submissions. Private scores sit close to CV, while public scores sit about 0.015 below it. CV also ranked the submissions correctly at the top: v10 has both the highest CV (0.7620) and the highest private score (0.7600). The submission with the best public score (0.74302) has the lowest private score among the stacks (0.7519).

8 Analysis

8.1 Why the public leaderboard disagreed with cross-validation

Our first stack submission (v8) scored 0.73843 on the public leaderboard, 0.014 below its CV, whereas both TF-IDF submissions had scored about 0.02 *above* theirs. We tested three explanations. (a) *Pseudo-label memorisation* (Section 8.2): stack v9, which has no pseudo-labelled members, scored 0.73848, the same as v8. (b) *Calibration shift*: test and OOF predictions have the same class mix and the same confidence distribution. (c) *A non-random public slice*: we scored our OOF predictions on the training posts inside every contiguous id window that holds 40% of the test posts, and compared the result with all three public scores at once.

Table 14: Matching the three public scores with OOF scores on candidate slices.

Training posts scored	Stack v9	TF-IDF	TF-IDF tuned	RMSE to public
Public leaderboard	0.7385	0.5409	0.5463	—
ids 0–24,428	0.7275	0.5396	0.5421	0.0068
ids 14,551–38,901	0.7437	0.5357	0.5376	0.0066
200 random 40% subsets				best 0.0077; median 0.0198

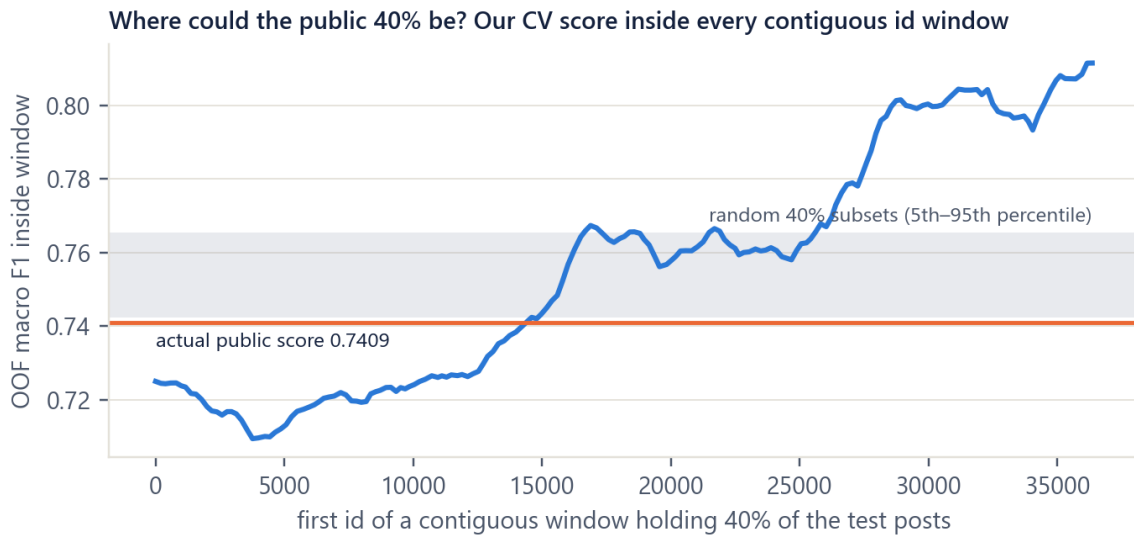


Figure 14: OOF macro F1 of the final system inside every contiguous id window holding 40% of the test posts, against the window's first id. The grey band is the 5th–95th percentile of random 40% subsets, and the orange line is this system's public score.

Contiguous windows reproduce all three public scores ($\text{RMSE} \approx 0.007$), and random subsets rarely do: fewer than 1 in 200 match as well. The public part of the test set is therefore most likely a contiguous low-to-middle id range, which is our hardest region (Figures 12 and 14). Attempts to adapt to it (region-specific multipliers or a stacker weighted towards low ids) lost 0.005–0.018 on held-out folds, so we kept selecting by CV. The private results (Section 7.6) confirm this choice.

8.2 Pseudo-label memorisation

Pseudo-labels helped single encoders (+0.01 to +0.03) but not the stack. A model trained on pseudo-labelled test posts partly memorises them: e5-large with pseudo-labels agrees with its own pseudo-labels on 87.6% of those test posts, against 77.4% for plain e5-large. The stacker learns its trust in each member from OOF predictions, where no such memorisation exists, and would over-trust the member on test. Because the public scores with and without pseudo-labelled members were identical, the final system excludes them.

8.3 Stacker variance

Re-running the same stacker with another seed or another scikit-learn build moved the held-out score between 0.7541 and 0.7580, an effect larger than most feature ideas we tried. Averaging five GBDT seeds per fold removes most of this variance, and it is the only difference between v9 and the final system. Averaging different stacker families (LightGBM [22], CatBoost [23], a feature-subsampled HistGBDT and two member variants; “grand6”) gave a similar, slightly larger gain (0.7576 OOF, 0.7594 held-out).

Table 15: Variants explored in the final hours. Most fall within the stacker’s own noise (± 0.003).

Variant (v9 members; single seed unless stated)	OOF tuned	Held-out
Baseline (scripts/stack.py)	0.7555	0.7580
+ labels of the 20 most similar training texts (char TF-IDF $\rightarrow$ SVD-256)	0.7522	0.7554
+ context-aware e5-large (0.626 alone) / instead of e5-large	0.7527 / 0.7538	0.7531 / 0.7544
Two-stage cascade (neutral vs toxic, then explicit vs subtle)	0.7496	0.7492
+ neighbour labels at positions $\pm 1 \dots 6$ with text similarity to each	0.7492	0.7504
Extra id windows ($k = 3, 50, 5,000$)	0.7572	0.7571
Slower GBDT / heavier regularisation	0.7527 / 0.7508	0.7540 / 0.7524
Spline features + logistic-regression stacker	0.7304	0.7331
LightGBM / CatBoost / XGBoost stacker	0.7567 / 0.7547 / 0.7539	0.7573 / 0.7551 / 0.7561
Stacker averaged over 5 seeds	0.7559	0.7574
grand6: mean of six stackers	0.7576	0.7594

8.4 Distance-weighted neighbour labels (stack v10)

Count windows treat the five nearest training posts alike, whether they are 1 or 40 ids away. Weighting each neighbour’s label by $\exp(-\text{gap}/\tau)$ over the 60 nearest training posts on each side, for $\tau = 2, 6$ and 20, and adding the log kernel mass, raised OOF by +0.0065 to +0.0082 on each of three stacker seeds (0.7620 OOF, 0.7625 held-out). Every id region improved, including the public-like low ids (0.7274 $\rightarrow$ 0.7337). A leak test (perturbing a training post’s own label leaves its own features unchanged) passed. This variant was submitted but not selected, and it achieved the best private score (0.7600).

8.5 Negative results

Table 16: Ideas that did not improve the system.

Idea	Effect (held-out, vs the stack of the time)
Pseudo-labelled members in the stack	same public score (0.73843 vs 0.73848); memorisation risk
Gemma full-data re-run replacing the 5-fold test mean	no gain
Labels of the most similar training texts	-0.003
Positional neighbour labels $\pm 1 \dots 6$ + text similarity	-0.008
HMM / sequence-smoothing features	-0.002
Context-aware e5-large added to / swapped into the stack	-0.005 / -0.004
Bangla specialists (BanglaBERT-large, MuRIL-large) added	-0.003
Two-stage cascade	-0.009
Heavier regularisation / logistic-regression stacker	-0.006 / -0.025
Region-specific multipliers or region-trained stackers	-0.005 to -0.018
Neighbours’ text-model probabilities as features	+0.0000, and leak risk
Language-run label mix minus own label	0.97 OOF: target leakage, discarded

9 Discussion

9.1 What the collection-order signal is

The id signal reflects how the dataset was assembled: posts collected together, from the same source, thread or period, tend to share labels. It is not a property of the language. Using it is within the rules, which only forbid external data, and the private leaderboard shows that it generalises to held-out posts drawn from the same collection. It would not generalise to a new collection, and it has no counterpart in a live system.

9.2 Deployable accuracy

A moderation system sees one post at a time, without its neighbours' labels, so the honest estimate of deployable accuracy is the text-only figure of about 0.63 macro F1, not 0.755. Reporting both numbers side by side is, in our view, necessary for any system that uses dataset structure.

9.3 Language disparity

Every model we trained is weaker on Bangla-script posts than on English and Banglish ones: 0.72 against 0.76 for the final stack, and 0.12–0.15 lower for the strongest text-only models (Gemma 4 31B: 0.516 against 0.662). The collection-order features narrow the gap considerably: a stacker on text probabilities alone scores 0.522 against 0.675, the full stacker 0.719 against 0.761. In a live setting without those features the larger gap would apply. Automatic removal decisions on Bangla content would therefore carry more false positives and false negatives. A responsible deployment would route low-confidence Bangla predictions to human reviewers and monitor error rates per language [3, 4].

9.4 Ambiguity of subtle toxicity

Near-identical posts disagree on their label about 16% of the time, and most errors of the final system fall between the explicit and subtle classes or between the subtle and neutral ones. Class-1 predictions are better used to inform human review than to trigger automatic penalties. The class weighting and macro-F1 calibration prevent the neutral class from being absorbed by the others, which in moderation terms protects ordinary speech from being flagged by default.

9.5 Limitations

- The results come from a single dataset collected by the organisers. The id signal, and hence most of the stack's gain, is specific to that collection.
- Several choices (member sets, windows) were selected among a handful of candidates on the same folds. We estimate the resulting optimism at about 0.002 and mitigate it with the held-out protocol.
- Gradient boosting differs slightly across scikit-learn builds (about 98% agreement on test predictions), and GPU training is not bit-for-bit deterministic.
- Private scores for three early submissions were not recorded. The public-slice analysis infers the split from three scores and could be wrong in detail.
- The competition window was about 24 hours. Context-aware Gemma, the most promising unexplored model, was not run because no suitable GPUs were free.

10 Conclusion

We presented a two-level system for three-class harmful-content classification in Bangla, English and Banglish. A fixed validation contract let a large number of heterogeneous runs be combined safely. Fine-tuned LLMs, Gemma 4 31B in particular, were the strongest text models, but most of the final score came from a structural property of the data: posts adjacent in collection order share labels. Exploited with leakage-safe features in a bagged gradient-boosting stacker, it lifts macro F1 from 0.63 to 0.755. Cross-validation predicted the private score to within 0.0003, while the public leaderboard, apparently a contiguous and harder slice, did not. For practitioners, our results argue for three habits: build the validation contract first, treat any score above the label-noise ceiling as a bug, and report deployable accuracy separately from leaderboard accuracy.

Reproducibility statement

The final submission is produced by `NOTEBOOKS/12_Stack_v9_Bag5_Submission.ipynb` from the saved level-1 predictions in a few minutes on a CPU. The notebook asserts 100% agreement with the submitted file, and we re-ran it for this report with identical fold scores (0.7407, 0.7505, 0.7409, 0.7459, 0.7501), OOF 0.7551 and held-out 0.7547. Level-1 training scripts (`kernels/_body.py`, `kernels/_llm.py`, `chpc/train_enc.py`, `chpc/train_llm.py`) and the exact code of every Kaggle kernel are in the repository. Appendix A lists all hyperparameters, and Appendix D gives the commands.

Acknowledgements

We thank the HerWILL and UAP organisers for the datathon, Kaggle for free GPU time on twelve team accounts, and our collaborator on the university cluster, who ran the large-model experiments.

References

- [1] T. Davidson, D. Warmusley, M. Macy, I. Weber. Automated Hate Speech Detection and the Problem of Offensive Language. *ICWSM*, 2017.
- [2] M. ElSherief, C. Ziems, D. Muchlinski, V. Anupindi, J. Seybolt, M. De Choudhury, D. Yang. Latent Hatred: A Benchmark for Understanding Implicit Hate Speech. *EMNLP*, 2021.
- [3] M. Sap, D. Card, S. Gabriel, Y. Choi, N. A. Smith. The Risk of Racial Bias in Hate Speech Detection. *ACL*, 2019.
- [4] D. Borkan, L. Dixon, J. Sorensen, N. Thain, L. Vasserman. Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. *WWW Companion*, 2019.
- [5] M. R. Karim, S. K. Dey, T. Islam, S. Sarker, M. H. Menon, K. Hossain, M. A. Hossain, S. Decker. DeepHateExplainer: Explainable Hate Speech Detection in Under-resourced Bengali Language. *IEEE DSAA*, 2021.
- [6] S. Khanuja, S. Dandapat, A. Srinivasan, S. Sitaram, M. Choudhury. GLUECoS: An Evaluation Benchmark for Code-Switched NLP. *ACL*, 2020.
- [7] A. Bhattacharjee, T. Hasan, W. U. Ahmad, K. S. Mubasshir, M. S. Islam, A. Iqbal, M. S. Rahman, R. Shahriyar. BanglaBERT: Language Model Pretraining and Benchmarks for Low-Resource Language Understanding Evaluation in Bangla. *Findings of NAACL*, 2022.
- [8] S. Khanuja et al. MuRIL: Multilingual Representations for Indian Languages. *arXiv:2103.10730*, 2021.
- [9] S. Doddapaneni et al. Towards Leaving No Indic Language Behind: Building Monolingual Corpora, Benchmark and Models for Indic Languages. *ACL*, 2023.
- [10] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, V. Stoyanov. Unsupervised Cross-lingual Representation Learning at Scale. *ACL*, 2020.
- [11] F. Barbieri, L. Espinosa Anke, J. Camacho-Collados. XLM-T: Multilingual Language Models in Twitter for Sentiment Analysis and Beyond. *LREC*, 2022.
- [12] P. He, J. Gao, W. Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. *ICLR*, 2023.
- [13] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, F. Wei. Multilingual E5 Text Embeddings: A Technical Report. *arXiv:2402.05672*, 2024.
- [14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.

- [15] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS*, 2023.
- [16] Qwen Team. Qwen3 Technical Report. *arXiv:2505.09388*, 2025.
- [17] Gemma Team, Google DeepMind. Gemma 4 open models; model cards *google/gemma-4-31B* and *google/gemma-4-31B-it*, 2026.
- [18] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023.
- [19] T. Wolf et al. Transformers: State-of-the-Art Natural Language Processing. *EMNLP (System Demonstrations)*, 2020.
- [20] D. H. Wolpert. Stacked Generalization. *Neural Networks* 5(2):241–259, 1992.
- [21] J. H. Friedman. Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics* 29(5):1189–1232, 2001.
- [22] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *NeurIPS*, 2017.
- [23] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin. CatBoost: Unbiased Boosting with Categorical Features. *NeurIPS*, 2018.
- [24] T. Chen, C. Guestrin. XGBoost: A Scalable Tree Boosting System. *KDD*, 2016.
- [25] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *JMLR* 12:2825–2830, 2011.
- [26] D.-H. Lee. Pseudo-Label: The Simple and Efficient Semi-Supervised Learning Method for Deep Neural Networks. *ICML Workshop on Challenges in Representation Learning*, 2013.
- [27] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman. Leakage in Data Mining: Formulation, Detection, and Avoidance. *ACM TKDD* 6(4), 2012.
- [28] Z. C. Lipton, C. Elkan, B. Naryanaswamy. Optimal Thresholding of Classifiers to Maximize F1 Measure. *ECML PKDD*, 2014.

Appendix A Hyperparameters of every level-1 run

Table 17: All level-1 runs. Batch is per device $\times$ devices (or $\times$ gradient-accumulation steps). *Effective step count halved by DataParallel (Section 5.3). All LoRA runs: r 16, α 32, dropout 0.05, all projections, warm-up 3%, cosine schedule. All encoders: AdamW, weight decay 0.01, warm-up 6%, cosine, class-weighted cross-entropy.

Tag	Model	Method	lr	Batch	Ep.	Len	Seeds	Hardware	Time
tfidf	—	word 1–2 + char 2–5 TF-IDF, LogReg C 4, balanced	—	—	—	—	—	CPU	s
xlmrb	xlm-roberta-base	full FT	2e-5	32 $\times$ 2*	3	128	42	Kaggle T4 $\times$ 2	127 min
mdeb	mdeberta-v3-base	full FT (fp32 load)	2e-5	32 $\times$ 2*	3	128	42	Kaggle T4 $\times$ 2	161 min
muril / muril5	muril-base-cased	full FT, 3 / 5 epochs	2e-5	32 $\times$ 2*	3 / 5	128	42	Kaggle T4 $\times$ 2	118 / 198 min
bnbert	banglabert	full FT + normaliser	2e-5	32 $\times$ 2*	3	128	42	Kaggle T4 $\times$ 2	110 min
twxlmr	twitter-xlm-r-base	full FT	2e-5	32 $\times$ 2*	3	128	42	Kaggle T4 $\times$ 2	129 min
xlmrL, xlmrL2	xlm-roberta-large	full FT, folds split over 2 accounts	1e-5	16 $\times$ 2*	3	128	42, 2	Kaggle T4 $\times$ 2	~430 min
indicb	IndicBERTv2-MLM-only	full FT	2e-5	16 $\times$ 2	3	128	42	Kaggle T4 $\times$ 2	155 min
murilPL, twxlmrPL, indicbPL	(as above)	full FT + pseudo-labels v1/v2	2e-5	16 $\times$ 2	3	128	42	Kaggle T4 $\times$ 2	170–189 min
xlmrLPL	xlm-roberta-large	full FT + pseudo-labels, one fold per account	1e-5	8 $\times$ 2	3	128	42	Kaggle T4 $\times$ 2	~135 min/fold
q14	Qwen3-14B	QLoRA nf4 + head (fp16 compute)	1e-4	16	1	192	42	Kaggle T4	~5.2 h/fold
xlmrL-s1337, -s7	xlm-roberta-large	full FT	1e-5	16	3	128	1337, 7	cluster	~12 min/fold
murilL	muril-large-cased	full FT	1e-5	16	3	128	42, 2, 3	cluster	~11 min/fold
bnbertL	banglabert_large	full FT + normaliser	1e-5	16	3	128	42, 2	cluster	~10 min/fold
me5L	mE5-large	full FT	1e-5	16	3	128	42, 2, 3	cluster	~12 min/fold
me5LPL	mE5-large	full FT + pseudo-labels v2	1e-5	16	3	128	1, 2	cluster	~14 min/fold
me5Lctx	mE5-large	text pair with neighbour text	1e-5	16	3	256	42, 2, 3	cluster	~18 min/fold
q14c	Qwen3-14B	QLoRA nf4 + head	1e-4	8 $\times$ 4	1	192	42, 2	cluster	~59 min/fold
q32	Qwen3-32B	QLoRA nf4 + head	5e-5	8 $\times$ 4	1	192	42, 2	cluster	115–132 min/fold
g31	gemma-4-31B	QLoRA nf4, label tokens	5e-5	8 $\times$ 4	1	128	42, 2, 3, 4	cluster	160–200 min/fold
g31PL	gemma-4-31B	label tokens + pseudo-labels v2	5e-5	8 $\times$ 4	1	128	1, 2	cluster	~240 min/fold
g31it	gemma-4-31B-it	QLoRA nf4, label tokens	5e-5	8 $\times$ 4	1	128	42	cluster	~225 min/fold
judge-g31	gemma-4-31B-it (FP8)	vLLM, 9-shot, label-token log-probs	—	—	—	2048 ctx	—	cluster	~44 min

Appendix B Per-fold scores of the final members

Table 18: Per-fold macro F1 of each seed run in the final stack (raw argmax). Seed runs are averaged within each member before stacking.

Seed run	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	OOF
g31	0.6297	0.6273	0.6232	0.6202	0.6188	0.6239
g31-s2	0.6246	0.6262	0.6213	0.6186	0.6163	0.6214
g31-s3	0.6196	0.6238	0.6215	0.6172	0.6249	0.6215
g31-s4	0.6237	0.6263	0.6191	0.6227	0.6201	0.6224
me5L-s1	0.5970	0.5961	0.5977	0.5914	0.5969	0.5958
me5L-s2	0.5913	0.5983	0.5940	0.5944	0.5883	0.5933
me5L-s3	0.5971	0.5966	0.5937	0.5898	0.5935	0.5942
q32	0.5959	0.5855	0.5918	0.5902	0.5866	0.5902
q32-s2	0.6041	0.5927	0.5940	0.5860	0.5991	0.5953
tfidf	0.5180	0.5233	0.5267	0.5240	0.5175	0.5219

Appendix C Development timeline

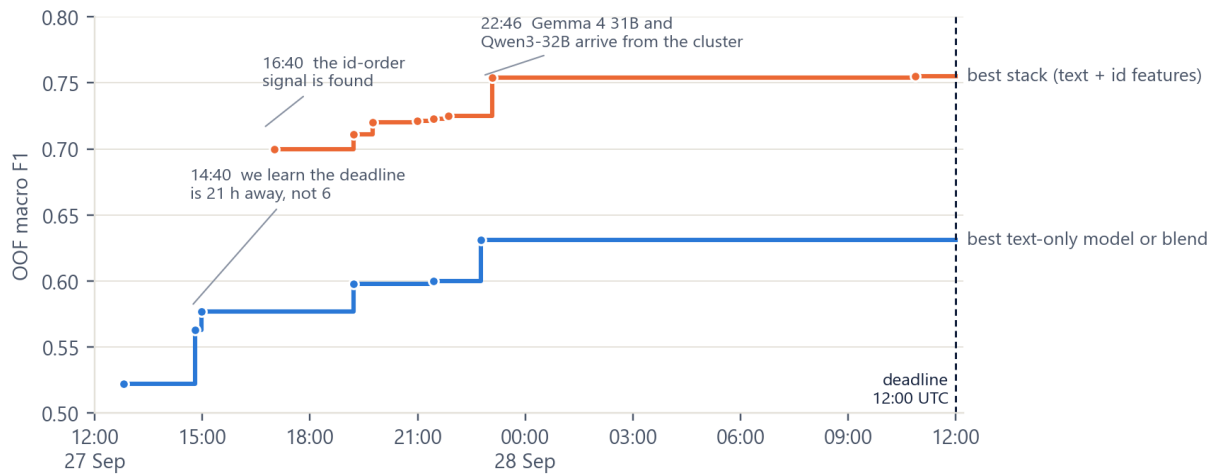


Figure 15: Best OOF macro F1 over the competition (UTC). Blue: text-only models and blends. Orange: stacks with collection-order features.

Table 19: Timeline of the project.

Time (UTC)	Event
27 Sep 12:30	Brief read; plan written (it assumed six hours). Data facts measured.
12:49	TF-IDF baseline submitted (OOF 0.5219, public 0.54091).
12:58	Wave 1: nine encoder kernels pushed across nine Kaggle accounts.
14:40	Deadline confirmed through the Kaggle API: 28 Sep 12:00 UTC.
15:47	Effective-step bug found (DataParallel halves steps).
16:40	Collection-order signal found (id-only kNN 0.600).
17:00	Stack v1: 0.700 OOF. Leaky feature variant tested and removed.
18:25	Fold-blind pseudo-labels v1; pseudo-labelled encoders pushed.
19:29	Qwen3-14B pilot clears its gate (0.5919 vs 0.5650).
19:45	Language-run leak (0.97) diagnosed and discarded; wide windows give stack v4 (0.7198).
21:52	Member selection: the best 5 beat all 12 (stack v7, 0.7253).
22:46	Cluster runs merged: Gemma 4 31B (0.624), e5-large, Qwen3-32B and others.
23:05	Stack v8: 0.7537 OOF.
28 Sep 07:56	First stack submission (v8): public 0.73843, below CV.
09:31	Stack v9 without pseudo-labels: public 0.73848 (same); public-slice analysis follows.
10:52–10:57	Final submissions: v9 5-seed bag, v9 + TF-IDF-context, v10.
12:00	Competition closes; third on the public leaderboard (0.74302).

Appendix D Repository and commands

Table 20: Repository layout.

Path	Content
NOTEBOOKS/12_Stack_v9_Bag5_Submission.ipynb	final system: loads member predictions, builds features, stacks, calibrates, writes and verifies the CSV
NOTEBOOKS/01–11	EDA, TF-IDF, encoder fine-tuning, LLM QLoRA, blending, id structure and stacking, pseudo-labelling, model comparison, stacks v8–v10
kernels/_body.py, kernels/_llm.py	Kaggle training scripts (encoders; Qwen3 QLoRA with one fold per GPU)
chpc/	cluster training (train_enc.py, train_llm.py, judge.py), Slurm queue, merge and export checks
scripts/	stack.py (stacker, pseudo-labels), blend.py, tfidf.py, push.py / pull.py (Kaggle orchestration), submit.py
build/out/	oof_ / test_ / log_ artifacts of every run; all submission files

```
# final submission from saved predictions (CPU, minutes)
jupyter nbconvert --to notebook --execute NOTEBOOKS/12_Stack_v9_Bag5_Submission.ipynb
# retrain members (GPU), one fold per process
python scripts/tfidf.py
python chpc/train_enc.py --model intfloat/multilingual-e5-large --tag me5L --fold 0
python chpc/train_llm.py --model google/gemma-4-31B --mode labeltok --tag g31 --fold 0 --quant nf4 --l
r 5e-5 --maxlen 128
python chpc/train_llm.py --model Qwen/Qwen3-32B --mode seqcls --tag q32 --fold 0 --quant nf4 --lr 5e-5
--maxlen 192
python chpc/merge.py g31
```